



22-CLAUDE-CODE-PROMPT-LIBRARY.md — Copy-paste prompts for all 30 phases



An agent drifts when it is asked vague questions. Every prompt here is written to remove the opportunity to guess: a named role, a named file, a named contract, and a required form of evidence.

1. The four rules of prompting this build

1. **Always assign a hat.** "Act as `SRC`" produces different, better code than an unassigned request — and it makes the two-hat review meaningful.
2. **Always name the source file.** "As specified in `15-CONCURRENCY-AND-JOB-GRAPH.md`" beats describing the requirement again and risking a contradiction.
3. **Always demand evidence.** "Report PASS/FAIL with command output" is the difference between a working phase and a confident summary of a broken one.
4. **Always forbid the shortcut.** End with the specific cheat you are worried about: *do not widen the tolerance*, *do not add `#[ignore]`*, *do not create a second SQLite writer*.

2. Session opener (paste at the start of every session)

Read in full: `CLAUDE.md`, `AGENT-TEAM.md`, `11-ERROR-TAXONOMY.md`, `12-DETERMINISM-CONTRACT.md`, `13-CONTRACT-FREEZE-AND-ADR.md`, `15-CONCURRENCY-AND-JOB-GRAPH.md`, `17-RESOURCE-BUDGETS.md`, `21-DEFINITION-OF-DONE.md`.

Then confirm back to me, in your own words: the 9 product invariants, the 5 reliability invariants, the banned patterns, and the rule about frozen contracts. Do not write any code yet. If anything in those documents contradicts anything else, list the contradictions before we start.

3. Phase kickoff

Open `phases/PHASE-NN-*.md`. Before writing code, produce:

1. The list of frozen contracts this phase reads, and the list it introduces.
 2. The error codes it may emit, with domain, severity and recovery for each.
 3. The budgets it must respect, from `17-RESOURCE-BUDGETS.md`.
 4. The determinism risks it introduces, from section 3 of `12-DETERMINISM-CONTRACT.md`.
 5. The FMEA rows from `14-FMEA-FAILURE-CATALOG.md` that apply.
 6. The task table from the phase spec, with the role assigned to each task.
- Wait for my approval of that plan. Do not start implementation.

4. Task implementation

Act as `<ROLE>` (`<full role name>`). Task `<N>` from `PHASE-NN` : `<task description>` .

Deliverable: `<exact files>` .

Constraints: copy the frozen contracts verbatim; no `unwrap` / `expect` / `panic` ; every failure returns a coded `AuraError` ; no unbounded channels; no non-deterministic constructs; add the budget `observe()` calls.

When done, list: files created, contracts touched, error codes added, budgets instrumented, tests written. Then stop — do not start the next task.

5. The two-hat review

Now act as `<DIFFERENT ROLE>`. You did not write this code and you are sceptical of it.

Run the adversarial pass from section 5 of `21-DEFINITION-OF-DONE.md`, all seven questions, with concrete answers. In particular: pick the test you think is weakest, delete the feature it covers, and show me whether the test actually fails.

Produce a findings list ranked by severity. Do not fix anything yet. If you find nothing, you have not looked hard enough — look again at error paths and at what happens on the second run.

6. Phase exit audit

Act as `QAL`. Audit `PHASE-NN` against `21-DEFINITION-OF-DONE.md` — the universal list plus the phase-type additions.

For every item: run the command, paste real output, mark PASS or FAIL. No "probably", no "should". Then produce the exit report using the template in section 4 of that file.

If anything fails, the verdict is NOT COMPLETE. Do not fix anything in this pass.

7. Specialist prompts

Contract change request

That is a frozen contract. Stop. Write `docs/adr/ADR-XXXX-<slug>.md` using the template in `13-CONTRACT-FREEZE-AND-ADR.md`, including a real grep-based blast radius listing every phase, crate, test and fixture affected. Then wait for my approval. Do not modify the contract or `contracts.lock` yet.

Determinism bug hunt

Act as `SRG`. Output differs between runs. Work through section 3 of `12-DETERMINISM-CONTRACT.md` in order and produce a table of file:line → suspected source → confidence. Then bisect by fixing thread count to 1 and re-testing. Report findings before changing anything.

Performance regression

Act as `PERF`. Stage `<name>` regressed `<X>` %. Profile before theorising. Give me a flame graph summary and the top 5 cost centres with percentages. Then propose fixes ranked by benefit-to-risk. Do not change the budget ceiling, and do not "optimise" by removing work — any speed-up must keep the golden manifest identical.

Model quality drop

Act as `MLL`. `model-gate` shows a regression. Report the per-slice table, identify which slices moved, and state whether this is a data, training, quantisation or EP issue with evidence for your conclusion. Check the leakage test and the calibration first. Do not lower a threshold.

Flaky test

Act as `QAL`. Test `<name>` is flaky. Find the actual source of non-determinism — timing, ordering, shared state, or a real race. Run it 200 times and report the failure rate. Fix the cause; do not add a retry, a sleep, or `#[ignore]`.

Crash triage

Act as `TLC`. Here is a run bundle. Read `run-summary.json` and `graph.json` first, then `errors.json`. Identify the failing task, its dependency chain, and the exact error code. Map it to an FMEA row in `14-FMEA-FAILURE-CATALOG.md`; if there is no row, write one. Then write the regression test **before** the fix.

Security review

Act as `SEC`. Review this phase against `19-OBSERVABILITY-AND-FORENSICS.md` and the security invariants. Specifically: does anything log a path, a filename, a person's name or a key? Does anything reach the network outside the governed gateway? Does anything upload imagery without the consent flag? Is any external text treated as an instruction rather than as data? Report findings with file:line.

UI work

Act as `SFE`. Implement `<component>`. Constraints: the UI thread must never block for more than 50 ms; the grid must virtualise; all IPC is typed and versioned; every error state renders the plain-English `user_message` plus a next action; every long operation shows honest progress and can be cancelled. Then act as `UX` and critique your own affordances against the review flows in the phase spec.

8. Prompts that cause damage — never use these

Do not say	Because	Say instead
"Make the tests pass"	Invites weakening assertions	"Find why the test fails and fix the cause"
"Just get it working"	Produces <code>unwrap()</code> everywhere	"Implement the task with the constraints in section 4"
"Refactor this to be cleaner"	Silently changes frozen contracts	"Refactor internals only; contracts must be byte-identical"
"Speed this up"	Removes work and calls it optimisation	"Speed this up; golden manifest must stay identical"
"Continue"	Skips phase exit	"Run the phase exit audit, then propose the next phase"
"Is this done?"	Invites an optimistic yes	"Audit against the DoD with command output"
"Add a quick workaround"	Becomes permanent	"Add a fix, or record debt with an owner and a date"

9. Context hygiene

- Re-paste the session opener whenever context is compacted or a new session starts. An agent that has forgotten the invariants will confidently violate them.
- Keep one phase per session where possible. Cross-phase sessions are where contract drift originates.
- After any long session, ask: *"List every frozen contract you modified in this session and show me `cargo xtask contracts --check`."*
- Before every commit: *"Run the stage-1 gates from `20-CI-PIPELINE-SPEC.md` locally and paste the output."*

10. The weekly ritual

Act as **EM**. Produce this week's status: phases completed with their exit reports, debt items opened and closed with ages, flaky tests and their quarantine status, budget trends versus baseline, model quality trends per slice, and the top three risks from **09-RISK-REGISTER.md** that moved. Then act as **CTO** and tell me the single most likely way this project fails in the next month, with the evidence for that judgement.